Microsoft

Git – Introduction

Module 7

Git setup

Scenario overview

- You are tired of having different versions of the same scripts running in your environment.
- You want to be able to leverage version control within your script repository.
- Your engineers are confused on updates and which version of a specific script or file they are supposed to be used.
- You want control over your scripts used in the production environment.

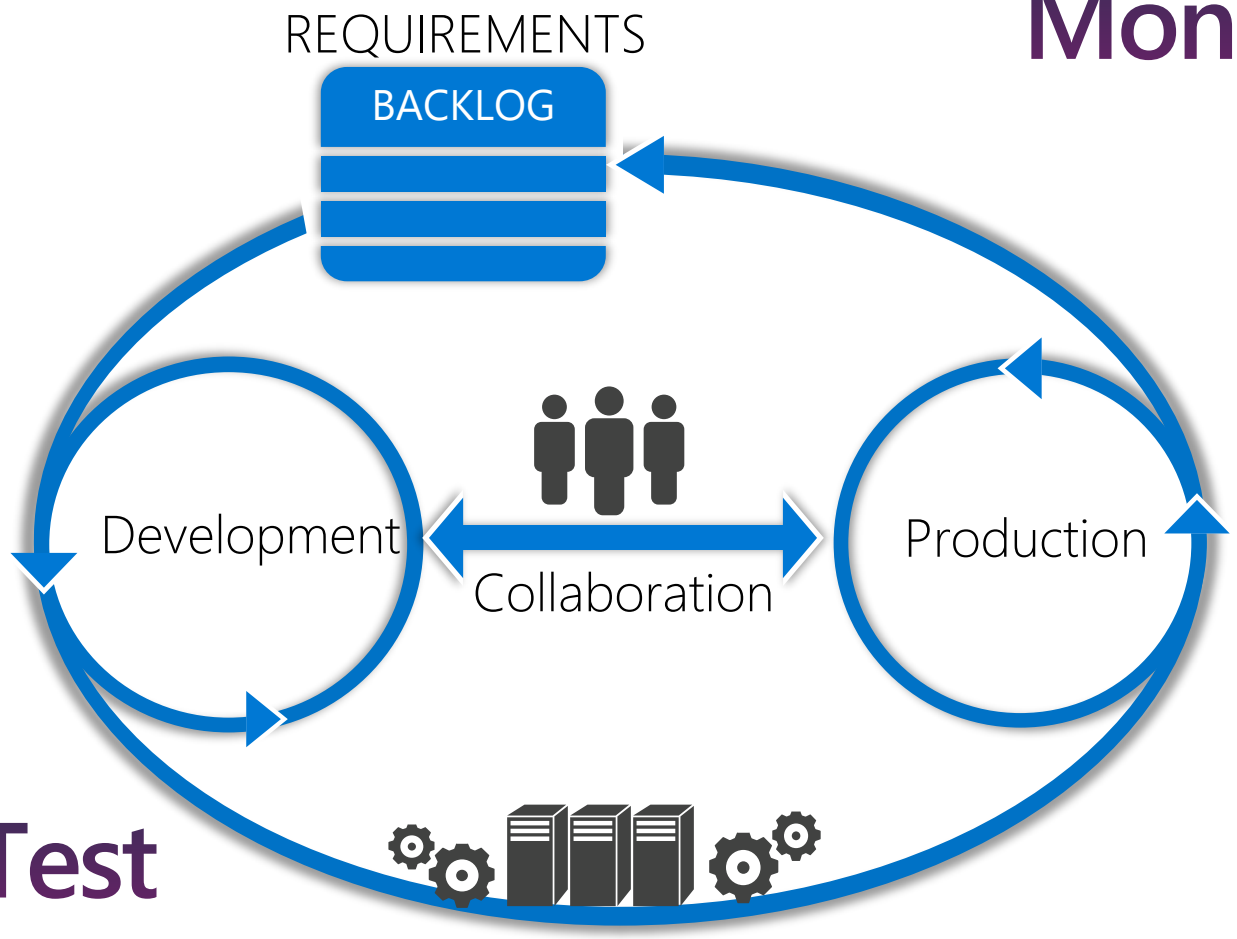
What is DevOps?

- The union of people, process, and products to enable continuous delivery of value to your end users.
- The contraction of “Dev” and “Ops” refers to replacing siloed Development and Operations to create multidisciplinary teams that work together with shared and efficient practices and tools.

DevOps – Deliver Faster, Smarter, and Continuously

Plan

Monitor + Learn



Develop + Test

Release

Introduction To Version / Source Control

Version control of scripts, files, applications, etc.

Records changes to a file or set of files over time

Allows file / project versioning

Local or centralized

Distributed version control change repository option

Easily swap versions

Ideal for teams

Undo changes

Stored on each machine

Types of Version Control

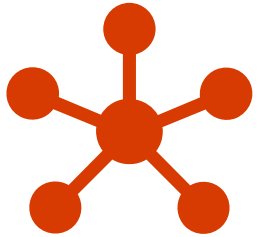
Distributed

Everyone has a copy

Changes made locally

Merged with an online central copy

Doesn’t generally require connectivity



Centralized

Master copy is stored centrally

Requires network connectivity

CI/CD strategy

Continuous Integration (CI)

- Team develops from same repository
- Limited drift from master branch
- Results contain few bugs
- Software works properly

Continuous Delivery (CD)

- Produce software/updates in short cycles
- Allows: building, testing, and releasing code with greater speed and frequency
- Reduces: cost, time, and risk of delivering changes by allowing for more incremental updates to applications/scripts

Source Control Management

Harness collaboration

Enable parallel development

Minimize integration debt

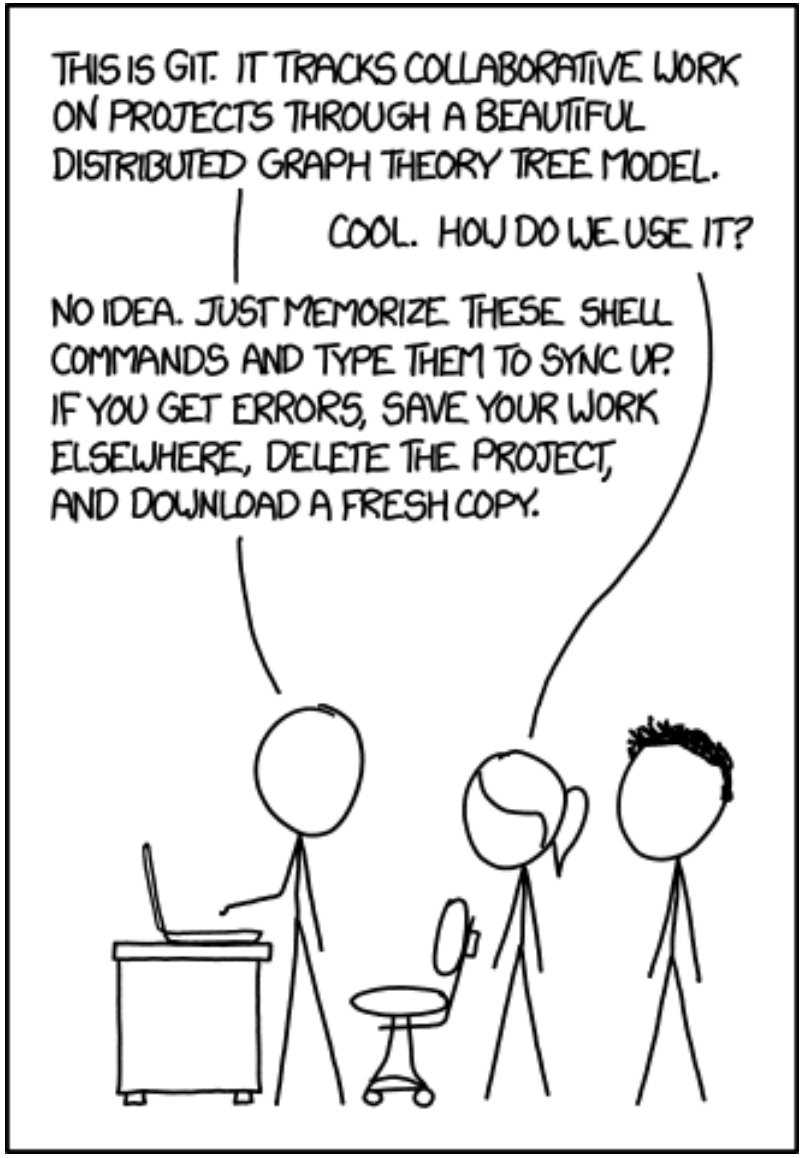
Act as a quality gate

What is Git?

- Distributed version control system
- Created in 2005 by Linus Torvalds
- Lightweight and fast
- Supports parallel development (branching)
- Fully distributed
- Open Source
- Integrated with many Content Index (CI) tools

Challenges of Git

- New for many operational engineers
- Command line based
- What's the point?
- Afraid to use Git
  - New toolset
  - Very powerful ("Dangerous")
  - Not user friendly
  - I don't want to become a developer



Git Commits

Retains each version by tracking ALL changes

Displays exact changes

Undo is holistic and reverts

Git Setup

Clone

- Create a new local repository to clone data into
- Copy existing repository to clone location
- Adds .git folder
- Adds remote origin
- Run with: *git clone ssh_url*

```
PS c:\> git clone https://github.com/anwaterh/MyNewRepository.git
Cloning into 'MyNewRepository'...
```

Git Structure

A Git repository is created by using git init

Git creates a .git folder

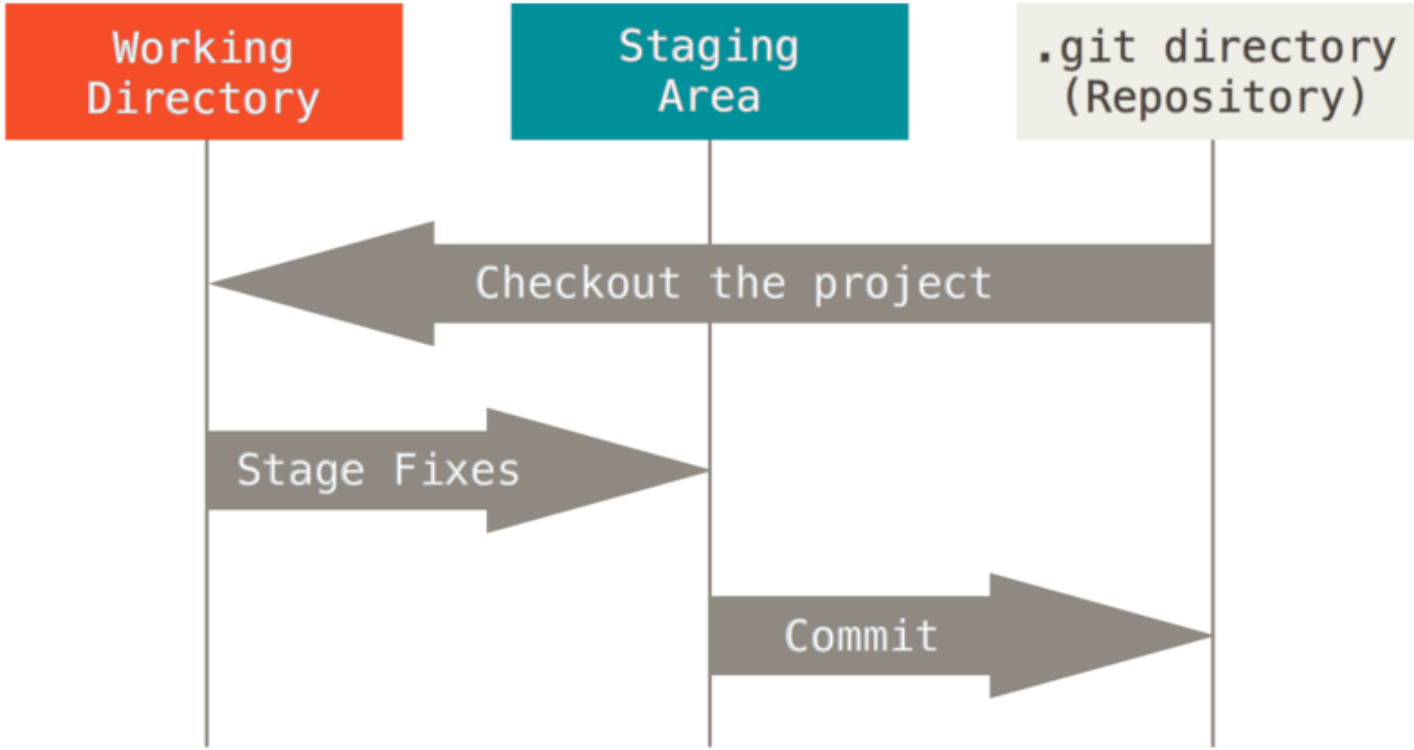
.git folder contains information needed for Git to function

To remove Git, remove the .git folder and retain all project files

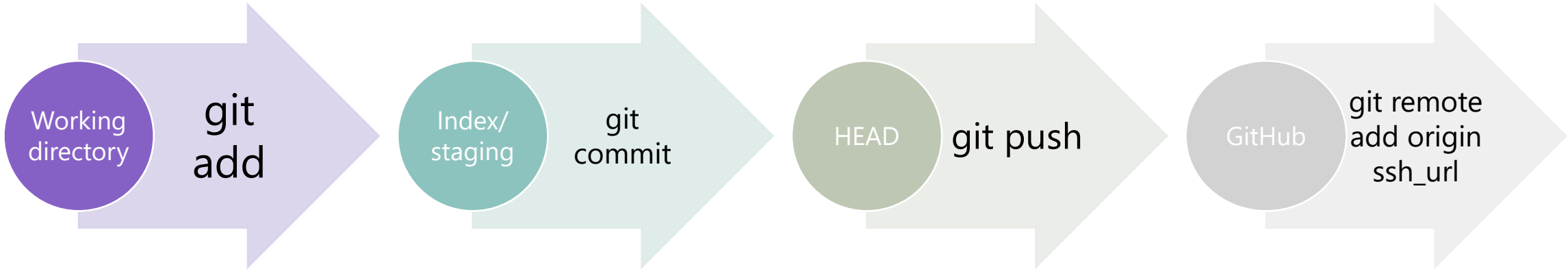
Git Layers

Three stages to track changes:

1. Working directory
2. Index/staging
3. HEAD



Git Process


Working directory
git add
Index/ staging
git commit
HEAD
git push
GitHub
git remote add origin ssh_url

Logging into a Git Repository

Repositories

Can be either hosted online or local (GitHub, GitHub Enterprise)

Represents the main project

Contains: files, commits, branches, and history for the entire project

Best practices:

Maintain a single source repository

Place all needed content for a project in a single repository

Minimize branches

Git vs. GitHub

- **Git**: a revision control system, a tool to manage your source code history
- **GitHub**: a hosting service for Git repositories, only stores code

Create Repository

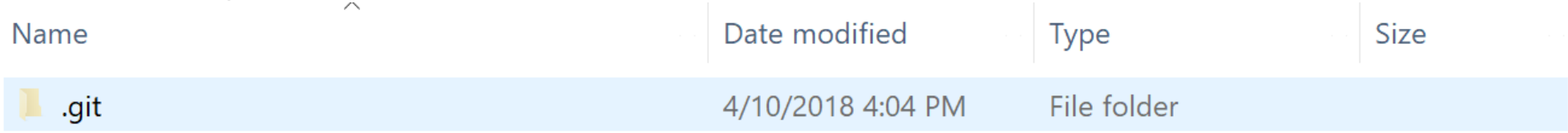
- **Create New** – *git init <RepositoryName>*

```
PS c:\> git init MyNewRepository
Initialized empty Git repository in C:/MyNewRepository/.git/
```

- **Clone from another location** – *git clone <location .git>*

```
PS c:\> git clone https://github.com/anwaterh/MyNewRepository.git
Cloning into 'MyNewRepository'...
```

- A local copy of source files is available



Workspace / Staging Area

- Files are created / edited in your workspace
- To track a file it first must be added to the staging area
- Staged files can then be committed (snapshot of the current staging area)
- All files / changes can be added – or only individual files

Adding Content to Staging Area

- **Add a file to staging area** – *git add <filename>*

```
PS c:\MyNewRepository> git add .\script.txt
PS c:\MyNewRepository> git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm –cached <file>... " to unstage)

        new file:     script.txt
```

- **Add all files to staging area** – *git add .*

Removing Staging Area Content

- Remove a file from staging (won't be part of the commit)

```
PS c:\MyNewRepository> git rm .\script.txt –cached
Rm 'script.txt'
PS c:\MyNewRepository> git status
On branch master

No commits yet

Untracked files:
   (use "git add <file>..." to include in what will be committed)

            script.txt

Nothing added to commit but untracked files present (use "git add" to
track)
```

Commit

- Snapshots the repository at that point
- All staged files become part of the commit
- Saves email address and username
- Contains a message describing the changes
- Commit has a unique hash which is used as a reference
- Use a text editor or the –m switch to provide the message

Committing Changes

- **Commit all changes** – *git commit –m "<commit message>"*

```
PS c:\MyNewRepository> git commit -m "My commit"
[master 67b2b30] My commit
 Date: Tue Apr 10 16:21:50 2018 +1000
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 script.txt
PS c:\MyNewRepository>
```

- Use the **–amend** switch to change the last commit

```
PS c:\MyNewRepository> git commit -m "Altered commit" --amend
[master 67b2b30] Altered commit
 Date: Tue Apr 10 16:21:50 2018 +1000
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 script.txt
PS c:\MyNewRepository>
```

Viewing History

Git log

```
PS c:\MyNewRepository> git log
Commit ebf96ff9862bc450a4b61c48742c0ec64639d1c (HEAD -> master)
Author: Anthony Watherston <anwather@contoso.com>
Date    Tue Apr 10 16:25:40 2018 +1000

    Made a small change

Commit 67b2b30e8cc6b8167ba947a55440730951ac44bq
Author: Anthony Watherston <anwather@contoso.com>
Date    Tue Apr 10 16:25:40 2018 +1000

     Altered commit
```

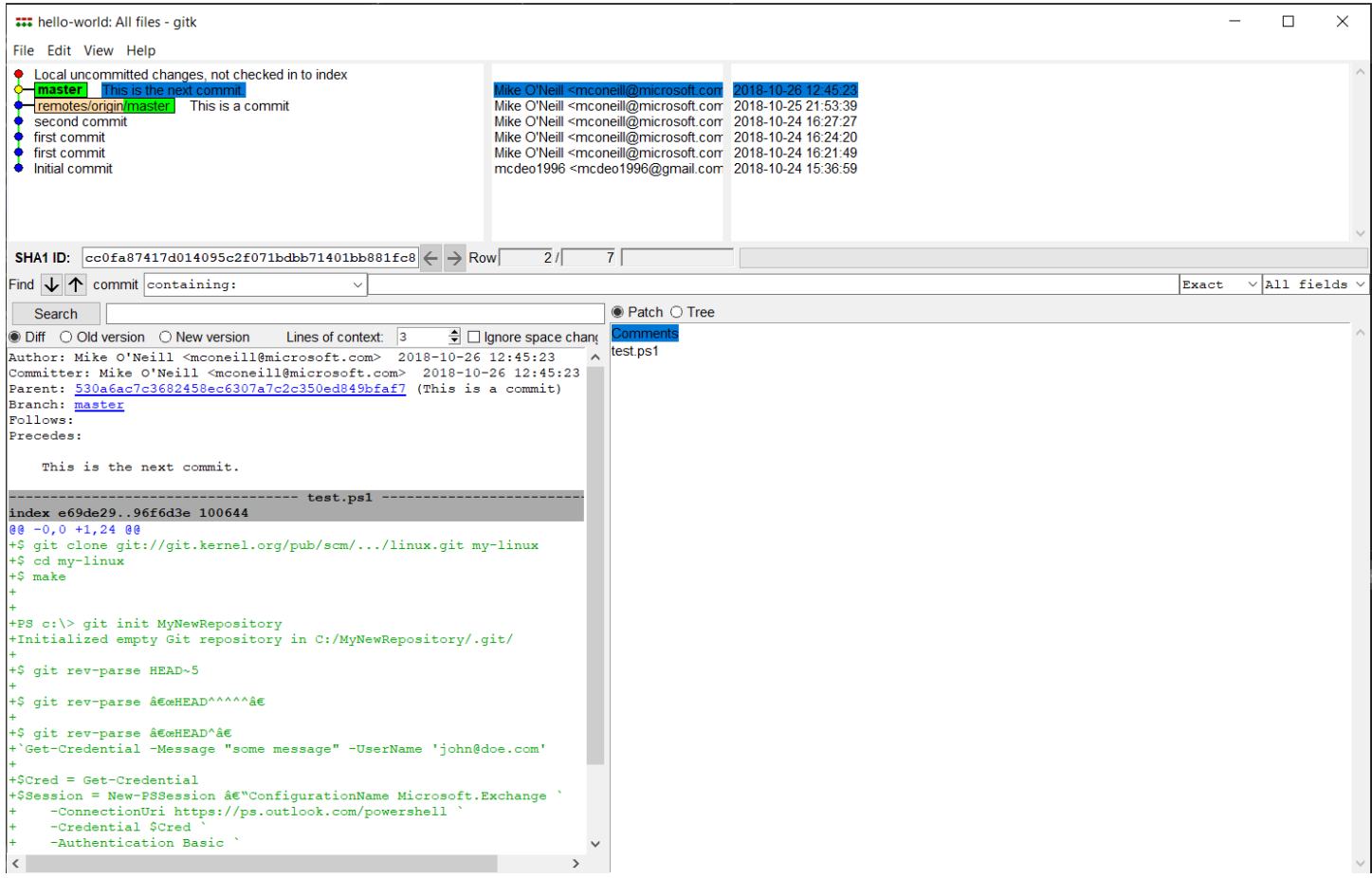
History Options

- Many options to condense output and add decorations

```
PS c:\MyNewRepository> git log --oneline
ebf96ff (HEAD -> master) Made a small change
Commit 67b2b30 Altered commit
PS c:\MyNewRepository> git log -oneline --graph
* ebf96ff (HEAD -> master) Made a small change
* Commit 67b2b30 Altered commit
PS c:\MyNewRepository>
```

History Options Cont.

Use **gitk** for a GUI history



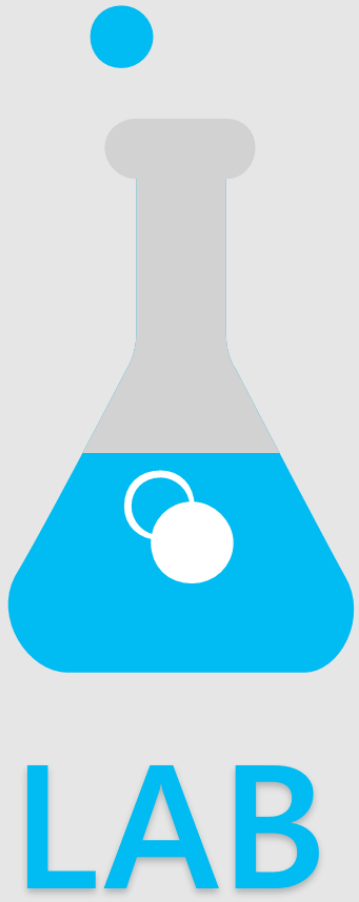
Demonstration

Advanced Features

Git – Introduction

(30 minutes)

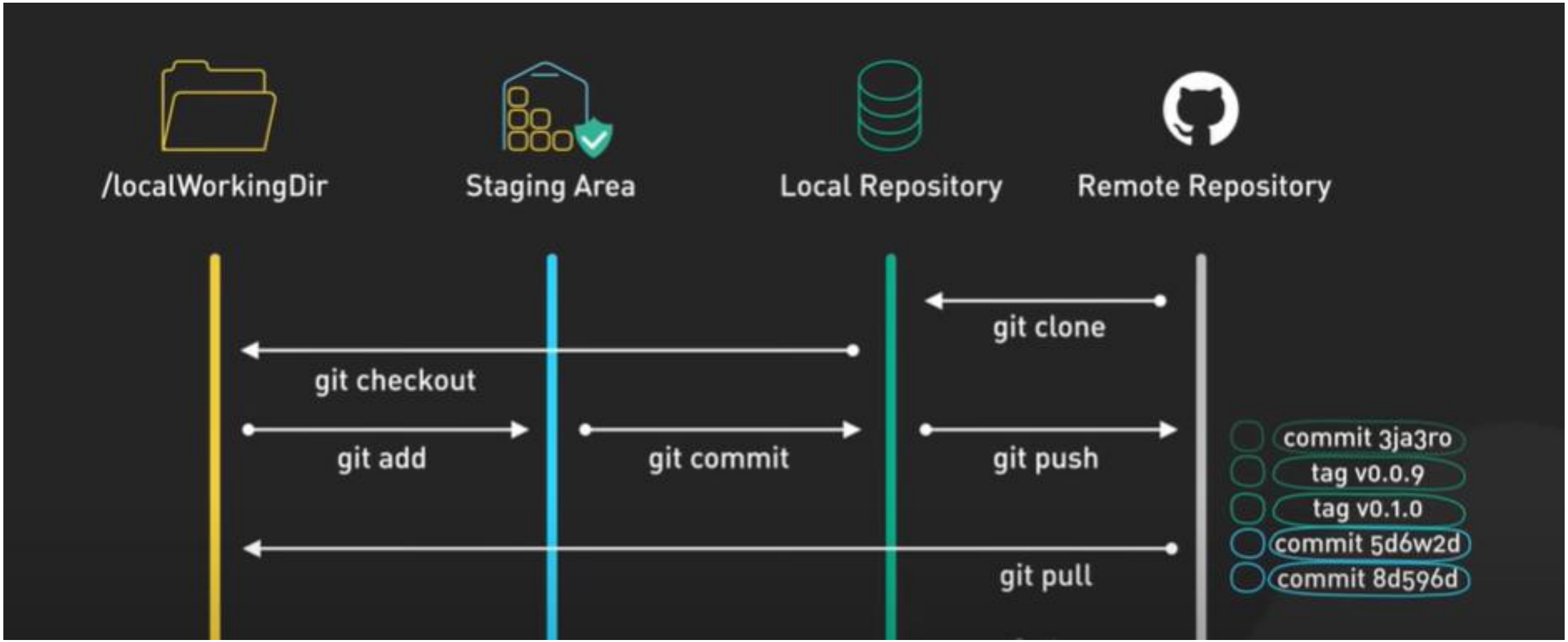
- Installing Git
- Setting up first repository



Why use version control?

- **Version control simplifies complex coding tasks**: It helps manage changes to your code, making it easier to add features and fix mistakes without confusion.
- **Time travel and alternate timelines**: You can take snapshots of your work, create branches for new features, and revert to previous versions if needed, much like traveling back in time or exploring alternate timelines.
- **Efficiency in development**: Using version control makes building, debugging, and deploying code more efficient by allowing you to manage and merge different versions seamlessly.

Git architecture



git init: Initialize a repository

- **Initialization of a Git repository:** The ***git init*** command is used to create a new Git repository in the working directory of your project.
- **Tracking changes:** Once initialized, Git monitors all changes in the directory, including the smallest modifications, and keeps a historical timeline of these changes.
- **Main branch creation:** The first branch created by Git is named "main," representing the main historical timeline of the project.

.gitignore: Ignore files

- **Purpose of .gitignore:** The ***.gitignore*** file is used to specify files and folders that Git should ignore and not track.
- **Usage:** You can list the names of files and folders in the ***.gitignore*** file to exclude them from version control.
- **Reversing Ignored Files:** If you want Git to start tracking a previously ignored file, simply remove its entry from the ***.gitignore*** file.

git add: Add changes

- **Staging Area:** The ***git add*** command is used to place changes into the staging area, preparing them to be committed to the project's history.
- **Selective Addition:** You can specify which files to add by naming them individually or add all changes using the -A option.
- **Controlled Commit:** This command allows you to pick and choose which changes to include in the next commit, giving you control over what gets recorded in the project history.
- **Remove content from staging area**: git rm .\script.txt –cached

git commit: Commit changes to memory

- **Purpose of git commit:** The ***git commit*** command is used to add changes from the staging area to the project's history, creating a snapshot of the current state.
- **Commit Messages:** When committing, you add a message with -m to describe the changes, making it easier to understand the project's history.
- **Frequent Commits:** It's better to commit changes often to ensure a detailed and manageable history of your project.

git status: Get the current status

- **Check Changes:** The ***git status*** command shows what changes have been made in your project since the last commit.
- **File Status:** It lists each changed file and indicates whether the changes are staged for commit or not.
- **Project Snapshot:** Running ***git status*** provides a snapshot of the current state of your project, helping you understand what needs to be committed or further modified.

git branch: Create an alternate timeline

- **Branches as Alternate Timelines:** Branches in Git allow you to create alternate timelines for your project, enabling you to develop new features or experiment without affecting the main codebase.
- **Parallel Development:** You can switch between branches to work on different versions of your code in parallel, maintaining a stable main branch while developing experimental features in separate branches.
- **Team Collaboration:** Different team members can work on their own branches, creating multiple timelines that can later be merged back into the main branch seamlessly.
- **GUI to visualize all branches**: gitk --all

HEAD: An introduction

- **HEAD Pointer:** The ***HEAD*** pointer in Git indicates your current position in the project's history, pointing to the latest commit on the active branch.
- **Branch Navigation:** When working with multiple branches, the ***HEAD*** pointer moves to the latest commit of the branch you're currently working on.
- **Time Machine Concept:** Think of the ***HEAD*** pointer as a time machine that allows you to navigate and work within different points in your project's timeline.

git checkout: Go to an alternate timeline

- **Branch Switching:** The ***git checkout*** command can be used to switch between branches, similar to the git switch command.
- **Commit Navigation:** You can use ***git checkout*** to navigate to any specific commit by providing its commit ID, allowing you to work on code from a previous point in time.
- **Interchangeability:** While ***git switch*** is a newer command, ***git checkout*** is still commonly used in documentation and tutorials for branch switching, and both commands are interchangeable for this purpose.

git switch: Go to an alternate timeline

- **Switching Branches:** The ***git switch*** command allows you to move between different branches in your project.
- **Conflict Handling:** If there are uncommitted changes that conflict with the branch you're switching to, Git will abort the switch to prevent data loss.
- **Resolving Conflicts:** To resolve conflicts, commit your changes on the current branch before attempting to switch to another branch.

DETACHED HEAD: An explanation

- **Detached Head Warning:** When you check out a commit in the middle of a branch, Git issues a "***Detached Head***" warning, indicating that you are working outside the main timeline.
- **Working in a Liminal Space:** In this state, you are not attached to any branch, allowing you to work on code from the past without affecting the current timeline.
- **Reattaching to a Timeline:** To continue working from this point, you can create a new branch using ***git branch***, which reattaches the detached head to the new branch, creating a safe alternate timeline for your changes.

The difference between switch and checkout

- **Separation of Concerns: *git switch*** and ***git checkout*** serve different purposes to avoid confusion. ***git switch*** is used only for switching between branches, while ***git checkout*** has multiple uses, including restoring files.
- **Restoring Files: *git restore*** is introduced to specifically handle restoring files to their last committed state, separating this function from ***git checkout***.
- **Preference and Clarity:** You can use either set of commands based on your preference, but using ***git switch*** and ***git restore*** can make your intentions clearer.

The difference between switch and checkout

- **Separation of Concerns: *git switch*** and ***git checkout*** serve different purposes to avoid confusion. ***git switch*** is used only for switching between branches, while ***git checkout*** has multiple uses, including restoring files.
- **Restoring Files: *git restore*** is introduced to specifically handle restoring files to their last committed state, separating this function from ***git checkout***.
- **Preference and Clarity:** You can use either set of commands based on your preference, but using ***git switch*** and ***git restore*** can make your intentions clearer.

git merge: Combine two timelines

- **Combining Branches:** The **git merge** command is used to combine changes from one branch into another, integrating different versions of your code.
- **Preservation of Branches:** Even after merging, the original branches remain intact, allowing you to continue working on them separately if needed.
- **Conflict Resolution:** During a merge, Git automatically combines matching pieces of code, but you may need to resolve conflicts manually if there are overlapping changes.

CONFLICT: How to fix merge conflicts

- **Conflict Identification:** When merging branches with conflicting changes, Git flags these conflicts and stops the merge process, requiring manual resolution.
- **Conflict Resolution:** Git highlights the conflicting code, marking it as "current change" (from your branch) and "incoming change" (from the branch being merged). You need to edit the file to resolve these conflicts.
- **Final Steps:** After resolving the conflicts, use ***git add*** to stage the resolved file and ***git commit*** to finalize the merge.